University Seminar Hall Booking System

DBMS Project Report | MySQL (RDBMS) + MySQL Document Store (NoSQL) + Python Streamlit

Team members (name / SRN)	
Guide	

1. The project in short

Students of our college request a seminar hall for a date and a time slot. The database checks the request, and if the hall is free it **holds** the hall for the student. The faculty member the student named as **event head** approves it first, then the **chairperson** of the student's department gives the final approval. When a booking is cancelled or rejected, the freed slot is **published to everyone**, so the hall does not stay empty. The college timings (9 AM to 4 PM, at most 2 hours, 30 minute gap) are enforced by the database itself.

Problem it solves. In our college halls are booked through the college website; the faculty in charge of the event and then the chairperson must approve. When someone cancels, nobody else finds out, so a hall that other students wanted stays empty. Double bookings and "which hall is free?" questions are also common.

Part	Technology	What it stores / does
RDBMS	MySQL 8.0 (InnoDB)	9 tables, 2 views, 3 stored procedures, 3 triggers, CHECK / UNIQUE / FOREIGN KEY constraints, transactions
NoSQL	MySQL Document Store (X DevAPI, port 33060)	2 collections of schemaless JSON documents: event_details and booking_logs
Front end	Python 3 + Streamlit	Login page first, then 3 roles: students (AIML PES1UG24AM001-100, CSE PES1UG24CS001-100) book and track requests; faculty (PES1UG24FAC001-012) approve events they head; chairpersons (PES1UG24CHR001-006) give the final approval and see dashboard, reports, NoSQL documents and manage
Sample data	sql/03_sample_data.sql	6 departments, 234 students, 12 faculty, 6 chairpersons, 7 halls, 3 slots, 202 booking requests (Aug - Dec 2026)

2. How the project meets the evaluation criteria

Criterion	What we have	Where to see it
1. Minimum 5 tables in the ER diagram	**9 entities / tables**, 12 relationships, Chen notation with cardinality (1:N, 1:1) and total/partial participation	Section 4, docs/er_diagram.html
2. RDBMS + NoSQL	MySQL tables for structured data with constraints and transactions; MySQL Document Store for event details and logs whose fields vary	Sections 5-7
3. Reports in the demo	**13 SQL reports** + 3 NoSQL reports with charts. R1 is the same idea as "average salary of every department": average attendance of every hall, for all months or one month	Section 8, Reports page
4. Novelty (our own)	Taken from how **our college** works: event head + chairperson approval, freed slots visible to everyone, college timing rules in the database	Section 3

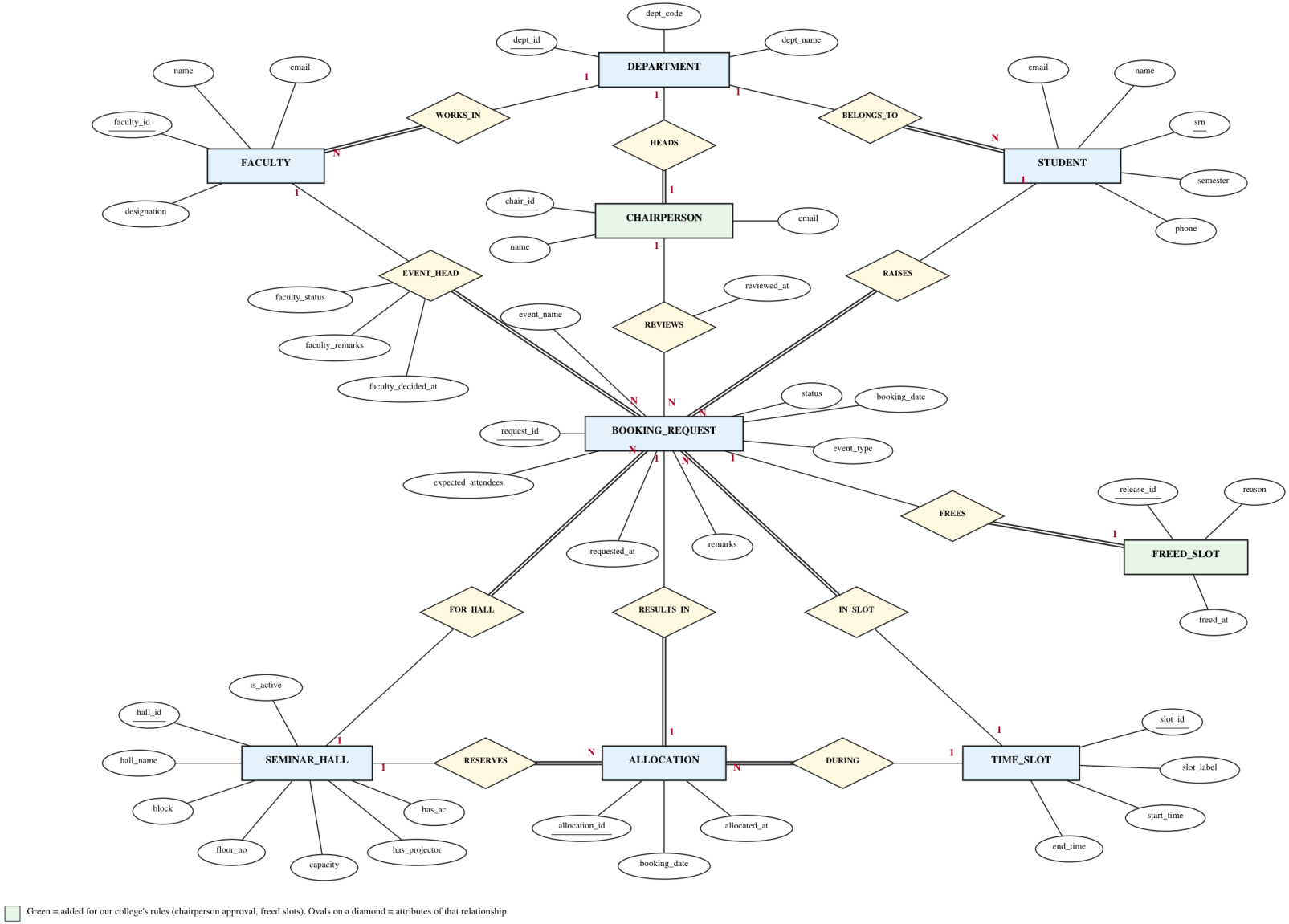
3. Novelty: from our own college

We asked how hall booking works in our college and built the database around it:

How our college works	What we built	Database objects
Halls are booked on the college website	Booking form, live hall x slot availability grid	request_booking procedure
The **faculty heading the event** must agree	A request first **holds** the hall. Only the faculty the student named as event head can approve or reject it; a rejection stops the request	faculty_review procedure, faculty_status / faculty_remarks / faculty_decided_at, report R13
The **chairperson** gives the final approval	One chairperson per department sees the student, the event head and the event head's answer, and approves or rejects	chairpersons table, review_booking procedure, reviewed_by / reviewed_at, report R12
If someone cancels, it should be visible to everyone (today nobody finds out)	Every cancelled / rejected booking is published as a freed slot. Everyone sees it on the dashboard; students who were refused that exact slot get a "free again" message	freed_slots table, trigger trg_free_hall_on_cancel, view v_open_freed_slots, report R11
Any department can book if the hall is free	No department restriction; only free, active and big enough	request_booking
Halls used 9 AM - 4 PM, max 2 hours, 30 min gap between slots	3 slots: 09:00-11:00, 11:30-13:30, 14:00-16:00. A wrong slot is refused by MySQL itself	CHECK chk_slot_hours, CHECK chk_slot_max_2h, triggers trg_slot_gap_insert / _update

Why it matters: without freed_slots, cancelling only deletes a row and the hall silently sits empty. Now the slot is recorded, shown to everyone, matched to students who wanted it, and measured: in the sample data 14 slots were freed and 7 of them were booked again by another student (report R11).

4. ER diagram (Chen notation)



Rectangles = entities, diamonds = relationships, ovals = attributes (underlined = key; ovals on a diamond belong to that relationship). Double line = total participation. Red 1 / N = cardinality. Green = added for our college's rules.

5. Relational schema (9 tables)

#	Table	Primary key	Foreign keys	Purpose
1	departments	dept_id	-	AIML, CSE, ECE, EEE, ME, BT
2	chairpersons	chair_id	dept_id (UNIQUE)	**New.** One per department; final approval
3	students	srn	dept_id	Students who raise requests
4	faculty	faculty_id	dept_id	Faculty who can be named as event head
5	seminar_halls	hall_id	-	Capacity, block, AC, projector, active flag
6	time_slots	slot_id	-	3 slots; college timing rules as CHECKs + trigger
7	booking_requests	request_id	srn, hall_id, slot_id, faculty_id (event head), reviewed_by (chairperson)	Every request: the event head's answer (faculty_status), the final status and reason
8	allocations	allocation_id	request_id, hall_id, slot_id	The hall schedule. **UNIQUE (hall_id, booking_date, slot_id)** makes double booking impossible
9	freed_slots	release_id	request_id	**New.** Every booked slot that became free again

Views: v_booking_details (one row per request with all names joined and a stage column: waiting for event head / waiting for chairperson / booked / rejected / cancelled) and v_open_freed_slots (freed slots that are still free and in the future).

Normalization: every table is in 3NF/BCNF - each non-key column depends only on its own table's key (e.g. the department name is not stored in students, only dept_id). hall_id, slot_id and booking_date are repeated in allocations on purpose, because a UNIQUE constraint cannot span two tables.

Constraints used

Constraint	Where	Why
PRIMARY KEY / FOREIGN KEY	all tables	identity, and no request for a student / hall / slot that does not exist
UNIQUE	dept_code, email, hall_name, (hall_id, booking_date, slot_id), ...	no duplicates, **no double booking**
CHECK	semester 1-8, capacity > 0, attendees > 0, end > start	reject invalid values
CHECK (college)	start >= 09:00 and end <= 16:00; end - start <= 2 hours	college timings
ENUM, NOT NULL, DEFAULT	status, event_type, requested_at	only allowed values, sensible defaults
ON DELETE CASCADE	allocations, freed_slots -> booking_requests	no orphan rows

6. Stored procedures and triggers (the rules live in MySQL)

request_booking (8 IN, 3 OUT parameters)

Runs inside START TRANSACTION ... COMMIT. Saves the request, then checks in order: date in the past, slot already started today, event head from another department, hall under maintenance, too many attendees, hall already taken. If all pass it inserts into allocations (the hall is held) and the request stays PENDING. A CONTINUE HANDLER for error 1062 turns a race between two students into a clean "already booked" rejection; an EXIT HANDLER rolls back on any other error.

```
CALL request_booking('PES1UG24AM001', 4, 3, '2026-11-27', 'Viva Demo',
                     'Seminar', 50, 4, @id, @status, @msg);   -- 4 = event head (Prof. Arjun Menon, AIML)
SELECT @id, @status, @msg;   -- PENDING | Hall held, waiting for the event head
```

faculty_review (event head) and review_booking (chairperson)

faculty_review locks the request (SELECT ... FOR UPDATE) and checks the caller is the event head named on the request and has not answered yet. Approve: faculty_status = APPROVED, now waiting for the chairperson. Reject: the request becomes REJECTED and stops. request_booking also refuses an event head from another department. **review_booking** checks the request is still PENDING and that the chairperson heads the student's department, then sets ALLOCATED or REJECTED with the reason, reviewed_by and reviewed_at. The chairperson may decide even before the event head answers.

Trigger trg_free_hall_on_cancel (AFTER UPDATE on booking_requests)

```
IF OLD.status IN ('PENDING','ALLOCATED') AND NEW.status IN ('CANCELLED','REJECTED') THEN
    DELETE FROM allocations WHERE request_id = NEW.request_id;
    IF ROW_COUNT() > 0 THEN            -- a hall was really held
        INSERT INTO freed_slots (request_id, reason) VALUES (NEW.request_id, NEW.remarks);
    END IF;
END IF;
```

The application only changes the status; the trigger frees the hall and publishes the slot, so no code can forget either step.

Triggers trg_slot_gap_insert / trg_slot_gap_update (BEFORE INSERT / UPDATE on time_slots)

A CHECK can only see one row, so the 30 minute gap between slots is checked by a trigger that compares the new slot with the existing ones and raises SIGNAL SQLSTATE '45000'.

7. Tested in MySQL (sql/05_demo_queries.sql)

We ran the demo script with the mysql client. The booking flow gave these results:

Step	Result returned by MySQL
Book Seminar Hall 1, 27 Nov, 14:00-16:00 (already booked)	REJECTED - Hall already booked for this date and slot
Book Conference Room (40 seats) for 60 people	REJECTED - Too many attendees: hall capacity is 40
AIML student names a CSE faculty member as event head	REJECTED - The event head must be a faculty member of your own department
Book AIML Seminar Hall, same slot, event head = faculty 4 (Prof. Arjun Menon, AIML)	PENDING - Hall held, waiting for the event head
Another AIML faculty member (faculty 3) tries to answer	Only the event head of this request can answer it
The event head approves	Approved by the event head, waiting for the chairperson
CSE chairperson (1) tries to decide (AIML student)	Only the chairperson of the student's department can decide
AIML chairperson (2) approves	ALLOCATED - Approved by the chairperson
Event head rejects request 190	REJECTED, stops there; trigger deleted the allocation and added a freed_slots row
Student cancels the approved booking	Allocation deleted, slot published in freed_slots

And the database refused every bad input:

Statement that must fail	MySQL error
Force a second booking into hall 2, 27 Nov, slot 3	1062 Duplicate entry for key 'uq_no_double_booking'
Add slot 15:00-17:00	3819 Check constraint 'chk_slot_hours' is violated
Add slot 09:00-12:00	3819 Check constraint 'chk_slot_max_2h' is violated
Add slot 11:00-11:20	1644 Slots must not overlap and need a 30 minute gap between them
Book for a student that does not exist	1452 foreign key constraint fails (transaction rolled back)
Add a student in semester 9	3819 Check constraint 'chk_semester' is violated

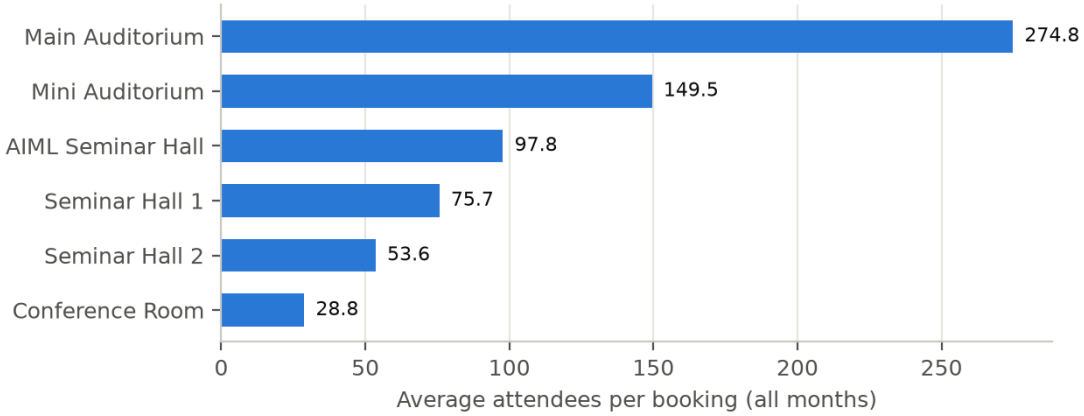
8. Reports

All 13 report queries are in sql/04_reports.sql. The website reads that same file, so every report runs the same SQL in MySQL Workbench and on the Reports page.

Report	SQL shown
R1 Average attendance of every hall	AVG, COUNT, GROUP BY
R2 Department-wise requests and success rate	COUNT(CASE WHEN ...), NULLIF, 3-table join
R3 Demand for each time slot	grouping on a lookup table, conditional count
R4 Month-wise trend / R5 Event types / R6 Top 5 students	DATE_FORMAT, AVG / MAX, ORDER BY ... LIMIT
R7 Events headed per faculty / R8 Rejection reasons	LEFT JOIN with condition in ON, GROUP BY text
R9 Halls busier than the average hall	nested subquery inside HAVING
R10 Busiest weekdays	DAYNAME, WEEKDAY
R11 Freed slots: were they booked again? (novelty)	correlated EXISTS inside COUNT(CASE ...)
R12 Chairperson decisions and average approval time	AVG(TIMESTAMPDIFF(...)) per department chairperson
R13 Event heads: how fast each faculty member answers	conditional counts + AVG(TIMESTAMPDIFF(...)) per faculty

R1. Average attendance of every hall (like "average salary of every department")

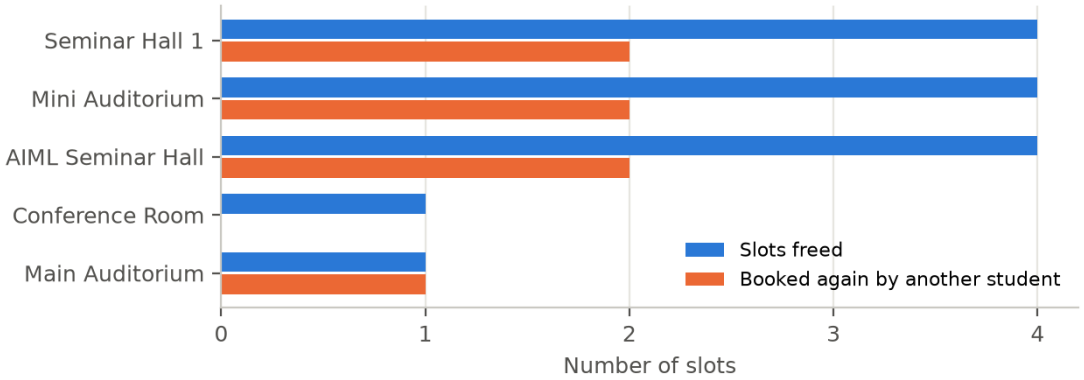
hall_name	All months	Aug	Sep	Oct	Nov
Main Auditorium	274.8	190.8	293.6	315.3	264.3
Seminar Hall 1	75.7	69.8	81.8	73.3	81.7
Seminar Hall 2	53.6	54.8	54.0	56.4	49.3
AIML Seminar Hall	97.8	96.2	93.1	99.6	115.5
Mini Auditorium	149.5	136.2	143.2	161.6	152.2
Conference Room	28.8	28.4	30.7	32.2	23.5



Average expected attendees at booked events. The Reports page has a month filter at the top: All shows every month, and clicking a month shows only that month. In SQL this is the variable @month.

R11. Freed slots and how many were booked again (novelty)

hall_name	slots_freed	booked_again	by_cancellation	by_rejection
Mini Auditorium	4	2	3	1
AIML Seminar Hall	4	2	3	1
Seminar Hall 1	4	2	3	1
Main Auditorium	1	1	1	0
Conference Room	1	0	0	1



R12. Chairperson decisions and average hours a student waited for the final decision

chairperson	dept_code	approved	rejected	waiting_for_chair	avg_hours_to_decide
Dr. Rajesh Iyer	CSE	47	1	0	11.8
Dr. Meera Krishnan	AIML	46	1	1	12.6
Dr. Sunita Hegde	ECE	20	1	1	13.8
Dr. Prakash Naidu	EEE	10	0	0	12.2
Dr. Vinod Kulkarni	ME	12	0	0	8.9
Dr. Anjali Deshpande	BT	24	0	1	14.6

R13. Event heads: requests received and average hours to answer (top 6)

event_head	dept_code	requests_received	approved	rejected	avg_hours_to_answer
Dr. Kavitha Rao	AIML	30	29	0	5.6
Prof. Naveen Shetty	CSE	28	26	1	5.4
Dr. Shalini Murthy	CSE	23	22	0	5.4
Prof. Arjun Menon	AIML	19	19	0	4.6
Prof. Kiran Joshi	BT	14	14	0	6.3
Dr. Pooja Reddy	BT	11	11	0	6.6

R2. Department-wise requests and success rate

dept_code	total_requests	allocated	pending	rejected	cancelled	success_rate_pct
AIML	63	42	2	15	4	66.7%
CSE	54	44	1	5	4	81.5%
BT	28	23	1	3	1	82.1%
ECE	27	20	1	6	0	74.1%
EEE	15	10	0	5	0	66.7%
ME	15	11	0	3	1	73.3%

9. NoSQL part: MySQL Document Store

MySQL 8 can also work as a document database: JSON documents in collections, used through the X DevAPI (add / find / modify / remove) on port 33060, with no SQL - the same model as MongoDB. All NoSQL code is in nosql.py.

Collection	What is stored	Why NoSQL fits
event_details	Extra details of each event: speaker, tools, club, prizes, tags, notes	Every event type has **different fields** (a lecture has a speaker, a workshop has tools, a hackathon has prizes). In SQL this needs many mostly-NULL columns
booking_logs	One entry per action: REQUEST, FACULTY_REVIEW (event head), CHAIR_REVIEW (chairperson), CANCEL	Append-only, high volume, and each action has a different shape

```
{"request_id": 5, "event_type": "Hackathon",
 "teams_expected": 25, "prizes": [{"position": 1, "amount": 10000}]}
{"request_id": 12, "event_type": "Guest Lecture",
 "speaker": {"name": "Mr. Farhan Ali", "organisation": "CloudNest Technologies"}}
```

Operation	X DevAPI call
Create	collection.add(doc) - on every booking, review and cancel
Read	find("event_type = :type AND :tag IN tags").bind(...)
Update	modify("request_id = :rid").array_append("tags", tag)
Delete	remove("_id = :id").bind("id", ...)
Aggregate (reports N1, N2)	find().fields("event_type", "COUNT(_id) AS documents").group_by("event_type")
Index	create_index on $.request_id

Each document stores the request_id of its MySQL row. One booking writes to **both databases**: the NoSQL part is saved in the background and no JSON is shown during booking, approval or cancelling. The chairperson's NoSQL page shows the same data as plain tables.

10. How the SQL is connected to Python and the Streamlit website

```
Browser (localhost:8501)  <-  app.py (Streamlit pages: form -> query -> table / chart)
                                |-- db.py     -> MySQL tables   (mysql-connector, port 3306)
                                |-- nosql.py  -> JSON documents (mysqlx, port 33060)
```

- **db.fetch_df(sql, params)** runs a SELECT and returns a pandas table that Streamlit draws with st.dataframe or a chart.
- **db.execute(sql, params)** runs INSERT / UPDATE / DELETE and calls conn.commit().
- **db.request_booking / db.faculty_review / db.review_booking** call the three stored procedures with cursor.callproc and read the OUT parameters (status and message).
- All user input goes through **%s placeholders**, so there is no SQL injection. The app logs in as seminar_app, a user with rights only on this database.
- Only the login page is shown until someone logs in. Students (AIML PES1UG24AM001-100, CSE PES1UG24CS001-100) see 4 pages, faculty PES1UG24FAC001-012 see 3, chairpersons PES1UG24CHR001-006 see 8; each ID is checked against its table. AIML: faculty FAC003-FAC004, chairperson CHR002; CSE: faculty FAC001-FAC002, chairperson CHR001. For now the password is the ID. The ID, role and department are kept in st.session_state, so everyone acts in their own name.
- A live counter at the top of every page re-reads MySQL every 5 seconds and pops up new requests and decisions, so the student, faculty and chairperson screens (even on two laptops) stay in sync.
- Slots that have already started today are never offered or shown as free: open_slots() keeps only start_time > CURTIME() for today, and the grid shows them as TIME OVER.
- Streamlit re-runs the page on every click: read the widget value, run the query, show the result. After a change, st.rerun() shows the new data immediately.

- Python never decides whether a hall is free; MySQL does. Python only sends SQL and shows the answer.

11. How to run

- Start the MySQL80 service; run sql/00_create_user.sql once as root.
- pip install -r requirements.txt and pip install --no-deps mysqlx-connector-python
- python setup_db.py (creates everything and loads the sample data; run again to reset)
- streamlit run app.py and open http://localhost:8501
- MySQL demo: open sql/05_demo_queries.sql in Workbench and run one query at a time (Ctrl+Enter).